Evolutionary Path Planning for Small UAS in Steady Wind Environments

Lucas Becker
Ann and H.J. Smead Aerospace
University of Colorado
Boulder, CO, USA
Lucas.Becker@colorado.edu

***Abstract*—Small Unmanned Aircraft Systems (sUAS) operating in high-wind environments face unique path planning challenges driven by Size, Weight, Power, and Cost (SWaP-C) constraints and susceptibility to ambient wind. This paper presents the implementation and evaluation of an Evolutionary Algorithm (EA) for path planning of a fixed-wing sUAS navigating through static obstacle fields in the presence of a steady ambient wind field. The algorithm optimizes simultaneously for throttle usage and time of flight while enforcing strict obstacle avoidance through a customized fitness function. The proposed approach is validated in simulation using the Ttwistor fixed-wing sUAS dynamic model within a MATLAB Guidance, Navigation, and Control (GNC) framework. A wind-aware cost function incorporating throttle penalty is benchmarked against a time-only baseline, with performance compared across throttle input, time of flight, and obstacle avoidance.**

I. Introduction

Unmanned Aircraft Systems (UAS) have seen rapid growth across both military and civilian applications, driving demand for increasingly capable autonomous navigation systems [8]. Among the most demanding operational scenarios are missions conducted by sUAS in complex, wind-affected environments, such as urban environments or atmospheric research in severe weather [10]. In these contexts, path planning—the task of generating a safe, feasible, and efficient route from a starting point to a target destination—becomes a critical capability.

The challenge of sUAS path planning is shaped by two sets of constraints. The first is computational: SWaP-C limitations restrict onboard flight computers to low-cost, lightweight hardware, placing hard bounds on the complexity of algorithms that can be run on these systems. The second is aerodynamic: due to their low mass and relatively slow cruise speeds, sUAS are disproportionately affected by ambient wind. In environments where wind speeds represent a significant fraction of vehicle airspeed, optimizing a path purely for shortest geometric distance can result in excessive control effort, energy depletion, or collision with obstacles [6] [11]. Urban environments further exacerbate this problem, as building geometry creates variable wind fields at the scale relevant to sUAS operations [9].

A broad range of algorithmic approaches have been applied to the autonomous path planning problem. Graph-search methods such as A* and D* provide optimality guarantees in discrete environments, but their computational complexity scales poorly with continuous state spaces and struggles to incorporate vehicle kinematics or wind-field dynamics without substantial overhead [2]. Potential field methods offer reactive, real-time obstacle avoidance but are prone to local minima and do not naturally optimize for energy efficiency [2]. More recently, deep reinforcement learning approaches have demonstrated impressive results in dynamic environments, but require extensive offline training and are difficult to deploy on resource-constrained embedded hardware [2].

Evolutionary Algorithms (EAs) have emerged as a practical alternative that occupies a favorable middle ground for this class of problem. EAs are population-based stochastic optimization methods that iteratively improve a set of candidate solutions through biologically-inspired operators such as mutation and crossover. They require no gradient information, handle non-linear cost functions naturally, and are readily parallelizable. A foundational EA-based planner for mobile robots operating in continuous two-dimensional environments encoded candidate routes as sequences of waypoints and applied mutation and selection operators to minimize path length while avoiding obstacles, establishing the core representation that later UAV planners would adopt [5]. This framework was subsequently adapted for fixed-wing UAV path planning, introducing segment-based path encoding and smooth arc-joining functions to satisfy the heading constraints inherent to fixed-wing flight [3] [4]. That work further incorporated probabilistic obstacle representations to produce trajectories robust to sensing uncertainty, demonstrating convergence to near-optimal solutions in Monte Carlo trials across varied obstacle densities [3]. The genetic algorithm framework was later extended to a multi-population model designed to support UAV re-planning under time-critical conditions. Maintaining several sub-populations evolving in parallel and exchanging individuals at defined migration intervals preserved diversity while accelerating convergence [1]. A comprehensive survey of subsequent development confirms that EA-based methods remain among the most actively developed approaches for UAV path planning, particularly for problems involving dynamic or uncertain environments [2]. Critically, that survey identifies fitness function design as a primary driver of solution quality, noting that most formulations optimize for path length and obstacle avoidance alone, with wind and energy considerations treated as secondary or absent objectives [2].

The environmental sensing literature provides important

context for the wind-aware planning problem. UAV-based in-situ sensor networks have been demonstrated to resolve urban wind fields at spatial resolutions finer than 10 meters, establishing the practical feasibility of pre-mission wind mapping at the scale relevant to sUAS operations [6]. Complementary advances in Coherent Doppler Wind Lidar (CDWL) have further improved the range resolution of remote wind sensing, supporting the generation of dense wind maps over urban domains prior to flight [7]. Together, these results motivate the assumption, adopted in this work, that a sufficiently accurate spatial wind map can be constructed before mission execution. Analysis of wind field estimation and its implications for urban UAV control has shown that even partial wind knowledge significantly reduces control effort compared to wind-unaware flight [11]. That finding directly motivates the present work: if exploiting known wind reduces energy expenditure during flight, then explicitly routing around headwind corridors during planning should yield further efficiency gains.

Despite this body of work, relatively few implementations explicitly couple the EA cost function with the aerodynamic cost of operating in a wind field. Most prior formulations optimize for path length and obstacle avoidance alone, treating wind as a disturbance to be rejected by the autopilot rather than a factor to be accounted for during planning. This paper addresses that gap by presenting an EA path planner intended for pre-mission ground-based use, whose fitness function directly penalizes the throttle effort required to fly each candidate trajectory through a spatially varying wind field.

The proposed algorithm generates an initial population of candidate paths defined by sequences of linearly connected waypoints, placed using a random sampling scheme that enforces monotonic progress toward the target. At each generation, paths are expanded through mutation and crossover operators, evaluated by simulating the full nonlinear aircraft dynamics of the Ttwistor sUAS through the wind field using a straight-line guidance law and feed-forward autopilot, and reduced back to the original population size through a fitness-weighted selection process. Because fitness evaluation requires simulating the full aircraft dynamics for every candidate path at every generation, the algorithm is designed to run offline before mission execution, producing a high-quality reference trajectory that accounts for both obstacle geometry and the ambient wind field. To isolate the contribution of the throttle term, results are compared against a baseline cost function that considers only time of flight and obstacle avoidance.

The algorithm operates under the assumption of a perfectly known environment: both the obstacle map and the wind field are treated as fully observable and static throughout the planning process. While modern CDWL systems and UAV-based sensor networks have demonstrated the ability to resolve urban wind fields at fine scales [6] [7], achieving a sufficiently accurate and complete pre-mission map of both wind and obstacles in practice remains a significant operational challenge that is outside the scope of this work.

The remainder of this paper is organized as follows. Section II describes the simulated environment, the Ttwistor aircraft model, and the full EA framework including population initialization, genetic operators, fitness evaluation, and selection. Section III presents simulation results comparing the throttle-penalized and time-optimized cost functions across path quality, control effort, and obstacle avoidance metrics. Section IV summarizes the findings and outlines directions for future work.

II. Methods

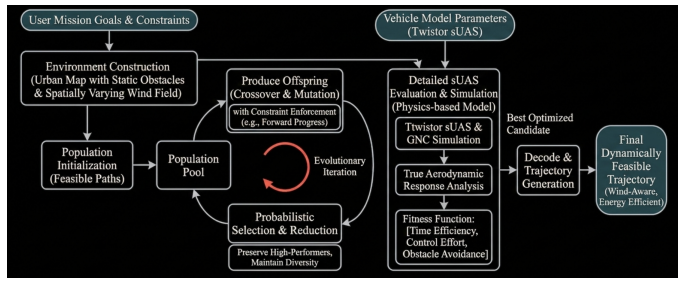

Fig. 1. Evolutionary Algorithm Block Diagram.

This section describes the complete framework used to evaluate the proposed evolutionary path planning algorithm. Fig. 1 describes the flow of this algorithm. The approach integrates a high-fidelity nonlinear aircraft simulation with a stochastic optimization process to generate wind-aware trajectories in a constrained environment. First, a representative urban-like environment is constructed, including both static obstacles and a spatially varying wind field. An initial population of feasible paths is then generated and iteratively improved through evolutionary operators, including mutation and crossover, while enforcing constraints on forward progress and feasibility. Next, a physics-based model of the Ttwistor sUAS and its associated Guidance, Navigation, and Control (GNC) architecture is used to simulate the true aerodynamic response of the vehicle along candidate paths. Each candidate path is evaluated using a fitness function that captures time efficiency, control effort, and obstacle avoidance. Finally, a probabilistic selection process reduces the population while preserving high-performing solutions and maintaining diversity. Together, these components form a closed-loop optimization framework capable of identifying dynamically feasible and energy-efficient trajectories in wind-affected environments.

A. Ttwistor Aircraft Model and GNC Simulation Framework

The Ttwistor is a small fixed-wing sUAS developed for atmospheric data collection in severe weather environments, including operations in close proximity to severe storms. All candidate path evaluations in the EA are performed by simulating the full nonlinear dynamics of the Ttwistor through the generated wind field, ensuring that the fitness scores reflect the true aerodynamic cost of each route rather than a geometric approximation.

The aircraft dynamics are modeled using the full nonlinear six-degree-of-freedom equations of motion. At each simulation time step, the aerodynamic model of the Ttwistor is used

to compute the forces and moments acting on the aircraft as a function of its current state and control inputs. The local inertial wind vector is sampled directly from the environment wind mesh at the aircraft's current grid position and transformed into the body frame before being applied to the aerodynamic model. This allows the forces and moments, and therefore the aircraft's dynamic response, to reflect the actual wind conditions along the trajectory. No sensor models, measurement noise, or state estimation are incorporated into the simulation; the aircraft has perfect knowledge of its own state at every time step, and the wind field is sampled exactly from the pre-computed environment map.

Path following is achieved using a straight-line vector field guidance law operating in the North-East-Down (NED) inertial frame. For each active path segment, the segment course angle χ_q is computed from the segment direction vector $\mathbf{q} = [q_n, q_e, q_d]^\top$ as:

$$\chi_q = \text{atan2}(q_e,\, q_n) \tag{1}$$

The aircraft's displacement from the segment origin $\mathbf{r} = [r_n, r_e, r_d]^\top$ is then resolved into along-track and cross-track components. Let $\delta p_n = x_n - r_n$ and $\delta p_e = x_e - r_e$ denote the North and East displacement of the aircraft from the segment start point. The along-track distance e_{px} and signed cross-track error e_{py} are computed by rotating this displacement vector into the segment frame:

$$\begin{bmatrix} e_{px} \\ e_{py} \end{bmatrix} = \begin{bmatrix} \cos\chi_q & \sin\chi_q \\ -\sin\chi_q & \cos\chi_q \end{bmatrix} \begin{bmatrix} \delta p_n \\ \delta p_e \end{bmatrix} \tag{2}$$

The signed cross-track error e_{py} is then used to generate the commanded course angle χ_c via an arctangent function:

$$\chi_c = \chi_q - \chi_\infty \cdot \frac{2}{\pi} \arctan(k_{path} \cdot e_{py}) \tag{3}$$

where χ_∞ is the maximum allowable course correction angle approached asymptotically as lateral error grows large, and $k_{path} > 0$ is a gain that controls the rate of convergence back to the segment. The following gains were chosen to create a balanced approach towards the line that minimizes overshoot while still approaching the line in a reasonable time:

$$\chi_\infty = 45^\circ, \qquad k_{path} = 0.01$$

As $e_{py} \to 0$ the commanded heading converges to the segment course angle χ_q, and as $|e_{py}| \to \infty$ the correction saturates at $\pm\chi_\infty$, preventing the guidance law from commanding unrealizable heading rates during large initial deviations. The guidance law is also capable of commanding a constant climb rate; however, this is not utilized in this simulation.

Attitude and throttle commands are executed by a feed-forward autopilot. The autopilot translates the desired course angle and airspeed produced by the guidance law into control surface deflections and throttle input. The throttle command at each time step is the primary quantity used in the fitness evaluation. This ensures that the throttle fitness term meaningfully distinguishes between routes that traverse headwind corridors, which require sustained high throttle to maintain airspeed, and routes that exploit tailwind or low-wind regions. The simulation is initialized at the trimmed steady-state airspeed V_{trim} and a fixed altitude, and is run at a fixed time step of $T_s = 1$ second for a maximum number of steps sufficient to cover the full domain at trim speed.

B. Creating The Environment

To evaluate the evolutionary path planning algorithm, a simulated two-dimensional operational environment was generated. The environment is represented as a discrete grid, constructed with a custom initialization algorithm. This methodology operates under the fundamental assumptions that the static environment is fully known and that the local wind field can be accurately measured and resolved at the spatial scale of the grid discretizations. Modern scanning Coherent Doppler Wind Lidar (CDWL) systems and localized networks of Unmanned Aerial Vehicle (UAV) in-situ sensors have been successfully demonstrated to map complex urban climates with spatial resolutions finer than 10 meters [6], [7].

For the simulations presented in this study, the domain encapsulates a region representative of the operational range of a small Unmanned Aircraft System (sUAS). Based on Department of Defense (DoD) Group 1 and Group 2 UAS classifications, as well as communication line-of-sight constraints, typical sUAS platforms conduct localized low-altitude missions within a confined environment [8] [10]. Consequently, the spatial domain is defined as a square region with a side length of $5000m$, broken into grid cells with a resolution of $10m/cell$.

The simulated environment incorporates two primary dynamic elements: static obstacles and localized wind fields, designed to replicate the challenging flight conditions of a dense urban environment. To model city infrastructure, 10 rectangular obstacles of varying dimensions—representing building footprints of different sizes—are distributed pseudo-randomly within the domain. To realistically represent urban environments and avoid bottlenecks, obstacles are placed with a strict buffer between them. This buffer ensures that corridors between adjacent structures remain open and effectively mimic urban canyons that influence wind fields [9].

The atmospheric conditions within these urban corridors are modeled as a steady-state, two-dimensional wind field. A background global wind vector is initially established using a random heading. To simulate realistic variations, Gaussian noise is generated on a coarse grid and mapped onto the high-resolution fine mesh via two-dimensional spline interpolation. The maximum wind speed was capped at $10m/s$ based on the operational constraints of the aircraft and what is seen as severe wind in an urban environment [11]. The resulting velocity vector components, V_x and V_y, are subsequently normalized and scaled such that the peak wind magnitude across the entire domain does not exceed a predefined maximum velocity threshold, V_{max}. To maintain physical consistency with urban fluid dynamics, wind vectors within any cell occupied by a static building obstacle are explicitly constrained to zero,

forcing the simulated flow around the structures. Using a defined random number generator seed allows this variable urban environment to be repeatable for comparisons. Fig. 2 and Fig. 3 represent two random generations of this environment following the size and wind parameters described above. For all simulations, the environment in Fig. 3 was used, with the starting point set to $(0, 0)$ and the target set to $(5000, 5000)$.

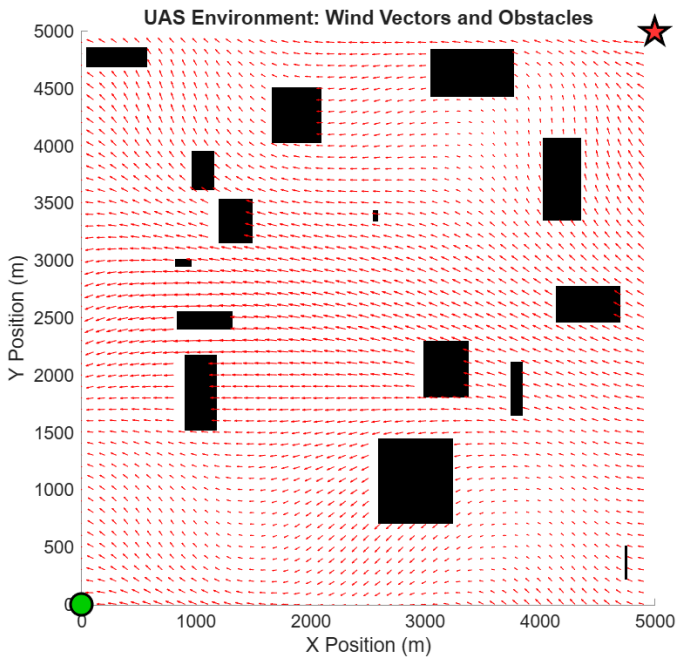

Fig. 2. Example of a generated $5000m \times 5000m$ operational environment. The vectors represent the interpolated wind field, while the solid blocks represent randomly placed static obstacles where the localized wind magnitude is constrained to zero.

C. Producing Initial Population

The initial population generation is a critical phase in evolutionary algorithms, balancing the need for genetic diversity with the goal of converging on the best solution [2]. To initialize the algorithm, a set of flight paths is constructed, where each candidate path is defined by a sequence of strategically placed waypoints connecting the start and target coordinates.

Drawing inspiration from constrained search space techniques [3], [4], the algorithm enforces forward progress by partitioning the straight-line distance between the origin and destination into a series of progressive distance bands. Waypoints are generated sequentially and are constrained to fall within their respective band, ensuring the simulated sUAS advances toward the target rather than wandering or backtracking.

To populate these bands, a probabilistic spatial sampling approach is utilized to ensure sufficient initial population diversity [1] [5]. For each waypoint, 80% of candidate locations are sampled from a Gaussian distribution centered along the direct line-of-sight diagonal, promoting general path efficiency. The remaining 20% of candidates are drawn from a uniform distribution across the entire operational domain. This

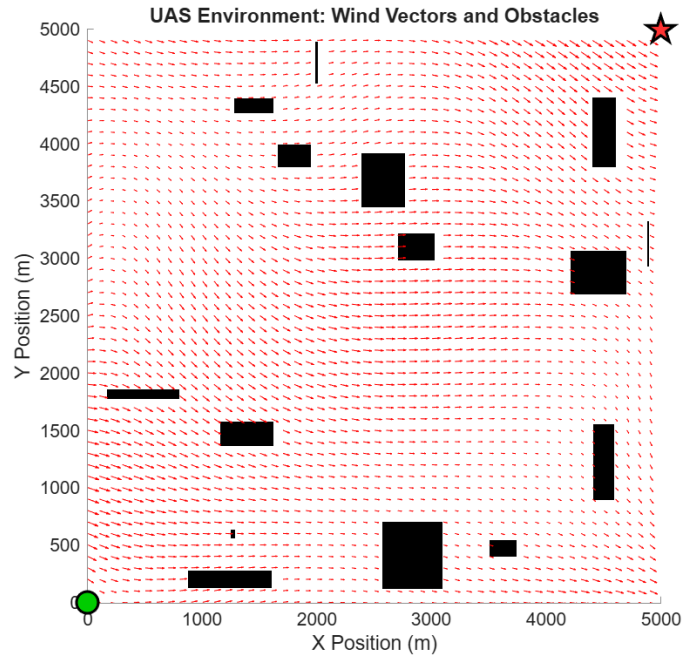

Fig. 3. A second instantiation of the environment generation algorithm utilizing a different random seed. This environment will be used for all future simulations in this paper.

uniform sampling injects highly exploratory paths into the initial population, which is critical for discovering paths around large obstacle clusters or severe wind zones. If a generated candidate falls within the allowable progressive distance band and domain boundaries, it is accepted. To prevent computation delays, a fallback mechanism ensures that if a valid point is not found within 500 attempts, a point along the direct diagonal is assigned.

Once the sparse waypoints are established for a candidate path, the route is linearly interpolated into a dense trajectory with $5m$ spacing between discrete points. An initial population of 40 paths is chosen to ensure a variety of paths are tested, and there is enough randomness in the population [5]. The initial population produced for the environment in Fig. 3 is shown now in Fig. 4

D. Mutating and Reproducing Population

At each generational step, the population undergoes genetic variation to explore the search space and exploit high-performing routes [2]. For each candidate in the current population, a probabilistic method determines whether the path will undergo mutation (40% probability) or reproduction (60% probability).

When reproduction is selected, a partner is chosen utilizing a fitness-proportionate selection mechanism [5]. In this approach, individuals are drawn from the population with a probability directly proportional to their evaluated fitness scores. This ensures that while higher-performing, energy-efficient paths dominate the breeding pool, lower-scoring paths still retain a probability of selection. Maintaining this weighted

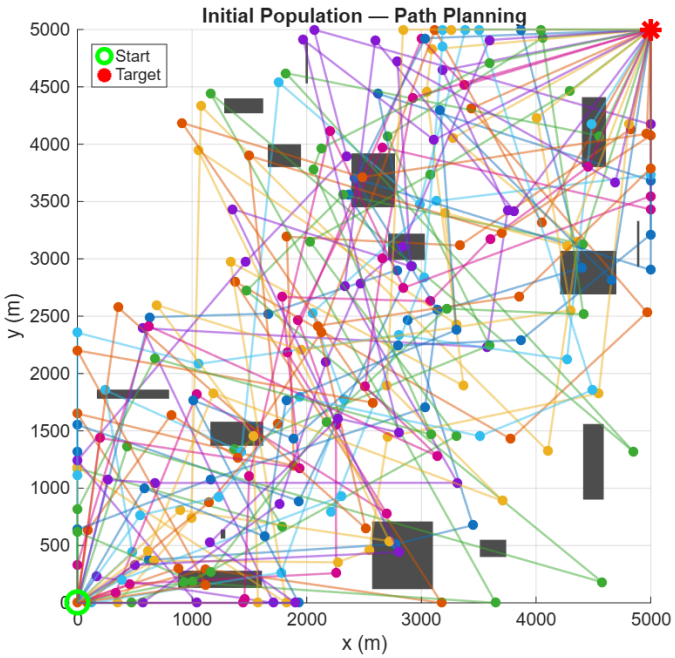

Fig. 4. Initial population of paths.

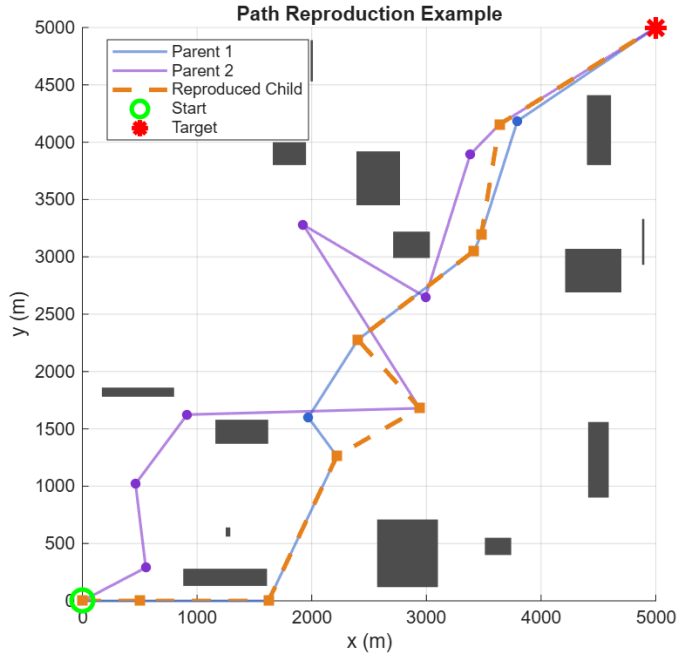

Fig. 5. Example reproduction between two paths.

inclusion is vital for preserving genetic diversity and preventing the population from prematurely converging on suboptimal paths [1].

The reproduction function uses a uniform crossover technique to exchange information between two parent paths. For every interior waypoint index, a 50% chance determines which parent's coordinates are inherited by the offspring. While uniform crossover maximizes genetic mixing, it can occasionally produce infeasible path segments where an inherited waypoint violates the constant progress constraint. To rectify this, a repair phase iterates through the newly formed sequence; any waypoint that does not meet the constraints is replaced by sampling valid candidate locations within its local neighborhood until the constraints are met. An example of reproduction between two paths is shown in Fig. 5

The mutation operator is designed to introduce localized diversity into the population [3], [4]. When a path is mutated, a single intermediate waypoint is randomly selected and perturbed within a localized search radius. This radius is scaled proportionally to the distance between its adjacent waypoints. To maintain the physical feasibility of the flight path, the new waypoint location must also satisfy the constraint of progression towards the target location described when generating the initial population. The same fallback method is applied here, where 500 attempts are made to place the new waypoint. If the new point is not placed correctly within these attempts, the mutation will not occur, and the path will remain unchanged. An example of a mutated path is shown in Fig. 6.

Following either genetic operation, the new waypoints are reconnected via straight-line segments. These segments are put into arrays matching the methods described with the initial

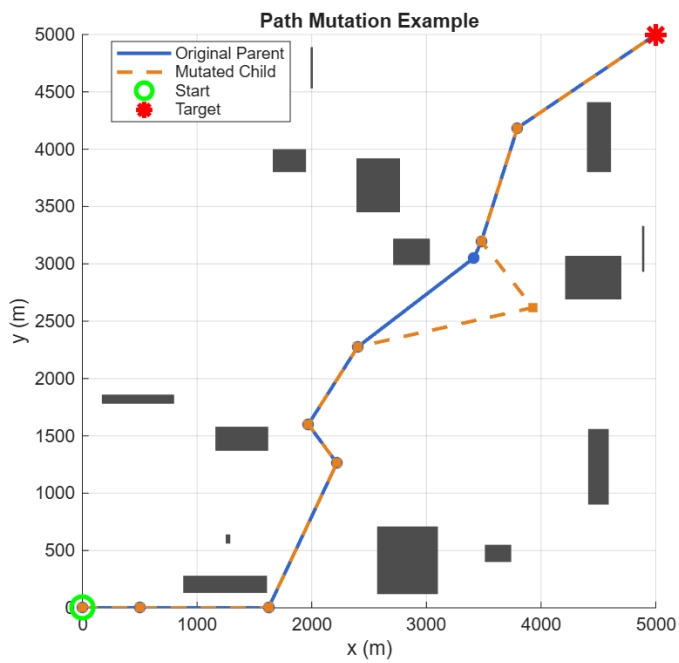

Fig. 6. Example mutation.

population.

E. Scoring

To evaluate the fitness of each candidate path, the custom MATLAB Guidance, Navigation, and Control (GNC) framework is utilized to simulate the Ttwistor's nonlinear dynamic response to the given paths in the population. Rather than evaluating paths purely on geometric distance, this approach measures the control effort required to navigate a dynamic urban wind field. To prevent unrealistic, uncoordinated overshoot at sharp waypoint transitions, a lookahead radius proportional to five seconds of trimmed flight ($5V_{trim}$) is defined. Once the vehicle reaches this lookahead radius, the guidance logic directs the aircraft to start following the next line segment. A sample trajectory is plotted against the waypoint paths in Fig. 7

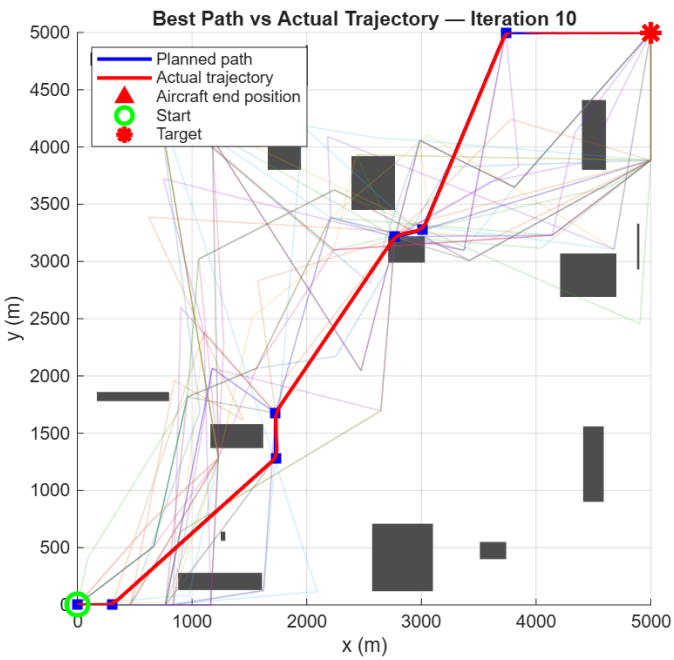

Fig. 7. The best population after one iteration is plotted with the actual path taken by the aircraft.

Upon reaching the target destination, the performance of the simulated trajectory is quantified using a fitness function. This function evaluates time of flight, throttle use, and obstacle avoidance, yielding a final fitness score derived from a weighted summation of three components:

1) **Time of Flight** ($w_{time} = 0.4$)**:** The time efficiency of the path is evaluated by tracking the total number of simulation iterations required to reach the target within a defined maximum step limit. The number of time steps is then normalized by the max steps allowed, resulting in values between 0 and 1. This normalization is important for comparison. Shorter, faster flights yield higher normalized scores. This objective commands the highest weighting in the composite evaluation.

2) **Control Effort/Throttle** ($w_{throttle} = 0.2$)**:** To assess energy constraints, the integral of the absolute throttle command is calculated over the entire flight path. This value is normalized against the worst-case maximum throttle use, where $\delta_t = 1$ for the entire flight. Including this metric directly penalizes trajectories that require excessive engine output to fight strong ambient headwinds. This encourages the algorithm to discover more efficient routes that exploit tailwinds or avoid heavy wind areas.

3) **Obstacle Avoidance** ($w_{obstacle} = 0.4$)**:** Safety is enforced via a severe penalty rather than a normalized 0 to 1 scale like the other components of the fitness function. At each time step, the aircraft's Cartesian position is mapped to the discrete environment grid. A completely collision-free path receives a nominal obstacle score of 1.0. However, any intersection with an obstacle triggers an immediate, heavy base penalty (-3.0), supplemented by a discrete per-step deduction (-0.1) for the duration the aircraft remains inside the structure. This ensures that paths lightly grazing a building corner are ranked above those flying directly through the center of multiple structures, guiding the algorithm away from hazards over successive generations. This approach is advantageous as it allows the population to learn the geometry of the obstacles while exploring the environment. Since this uses the aircraft's actual trajectory and not the path of straight lines, it will track collisions caused by imperfect path following as well.

The composite fitness score F for a candidate path is computed as a weighted sum of the three normalized component scores:

$$F = w_{time} \cdot S_{time} + w_{throttle} \cdot S_{throttle} + w_{obstacle} \cdot S_{obstacle} \tag{4}$$

where S_{time}, $S_{throttle}$, and $S_{obstacle}$ are the normalized scores for time of flight, throttle usage, and obstacle avoidance respectively, and the weights satisfy $w_{time} + w_{throttle} = 1$ when no collision occurs, with $w_{obstacle}$ acting as an additive penalty term. The weight assignment of $w_{obstacle} = 0.4$ was selected to ensure that obstacle avoidance dominated early generations, preventing the population from converging on fast but geometrically infeasible routes before the path topology had been established. The throttle weight $w_{throttle} = 0.2$ was calibrated empirically: values below this threshold produced paths behaviorally indistinguishable from the time-only baseline, while values above approximately 0.3 caused the algorithm to favor excessively long detours that substantially degraded time efficiency. The time weight $w_{time} = 0.4$ reflects the remaining priority once obstacle safety and energy efficiency are satisfied.

F. Reduction

Once the entire population is simulated and scored, the next step is to reduce the population back to its original size. This is done through a weighted selection that favors higher-scoring paths, but also allows for some lower-scoring paths to be preserved, which maintains randomness in the population. The weighting allows the best path to always remain in the population and the worst path to always be removed from the population. To assign valid survival probabilities, the raw

fitness scores must be normalized. Let S represent the set of fitness scores for the middle candidates. To prevent negative probabilities and ensure the lowest-scoring candidates still maintain a marginal chance of survival, the scores are shifted relative to the minimum score in S. The probability P_i of selecting candidate i with score $s_i \in S$ is defined as:

$$P_i = \frac{s_i - \min(S) + \epsilon}{\sum_{j \in S} (s_j - \min(S) + \epsilon)} \tag{5}$$

where $\epsilon = 10^{-6}$ is a sufficiently small constant added to prevent division by zero and ensure non-zero weights for all candidates in the selection pool.

III. Results

A. Simulation 1: Cost Function with Throttle Penalty

The initial simulation evaluated the evolutionary path planning algorithm using a comprehensive cost function that weighed flight time, obstacle avoidance, and throttle usage. In addition to the results presented here, the algorithm was tested across multiple randomly generated environments using different random seeds. While those runs are not explicitly shown due to computational time constraints, the qualitative behavior and convergence trends remained consistent across all tested environments.

As illustrated in Fig. 8, the algorithm is able to find a path that avoids all obstacles by iteration 20. However, this path is very inefficient in both time and throttle use. At this point the algorithm still needs more time to explore the space and refine a viable solution.

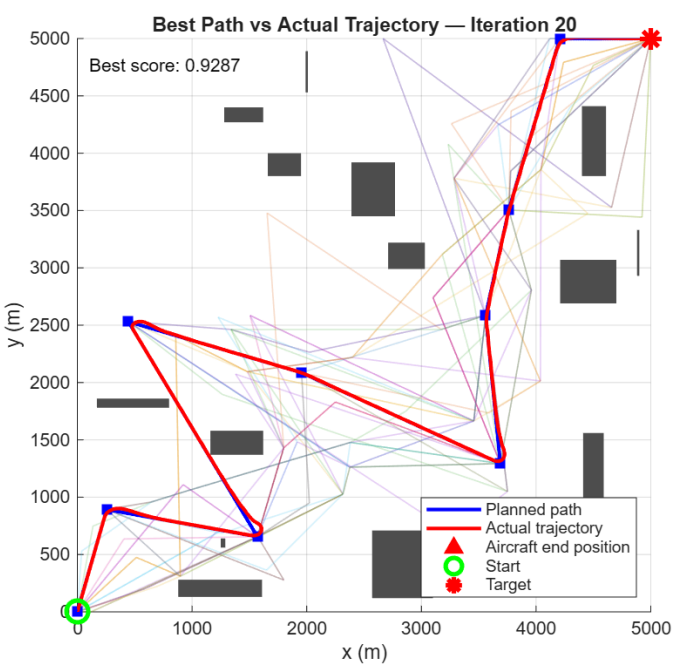

Fig. 8. Population and best path after 20 iterations.

Progressing to the 40th iteration (Fig. 9), the algorithm only sees a marginal increase in total score. However, the new best path is much smoother and more efficient. At this point, the algorithm begins to converge toward a stable solution. Across all simulations, it was consistently observed that the most significant improvements occur within the first 30-40 iterations, after which the solution changes only minimally.

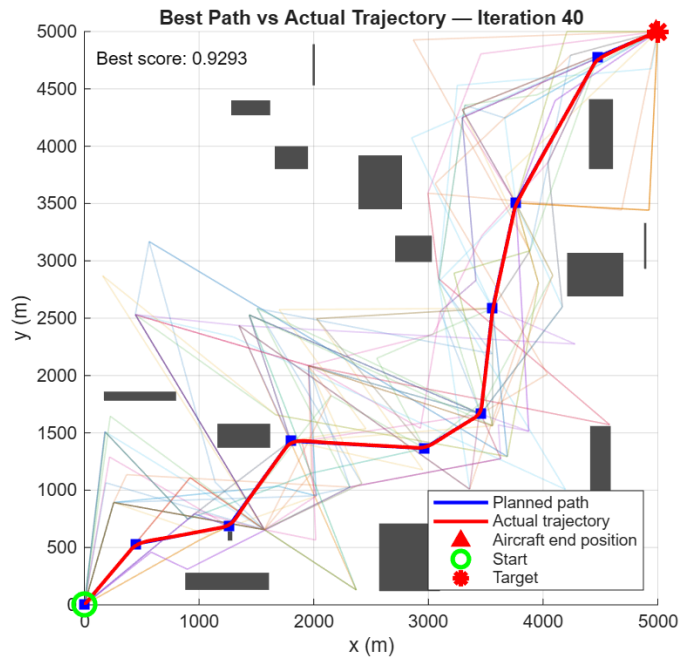

Fig. 9. Population and best path after 40 iterations.

As the evolutionary process continues through the 60th iteration (Fig. 10), the best path remains effectively unchanged from the 40th iteration. The population begins to cluster tightly around this solution, indicating that the algorithm has largely converged and is primarily performing minor local refinements rather than exploring new regions of the search space.

The simulation ends at the 80th iteration (Fig. 11). While the final path follows a very similar route to earlier iterations, small adjustments in waypoint placement produce slight improvements in the overall fitness score. The most noticeable change occurs in the final waypoint prior to the target, which moves closer to a nearby obstacle. While this improves efficiency, it also highlights a limitation of the current formulation: the algorithm tends to favor paths with minimal clearance from obstacles when no explicit buffer penalty is included.

The specific performance metrics for the best path at the conclusion of this 80-iteration simulation are detailed in Table I. The aircraft achieved a perfect obstacle avoidance score while maintaining high efficiency in both time and throttle usage.

TABLE I
Best Path Scores With Cost Function 1 (Throttle Included).

Best Score	Time Score	Throttle Score	Obstacle Score
0.9803	0.9966	0.8198	1.0000

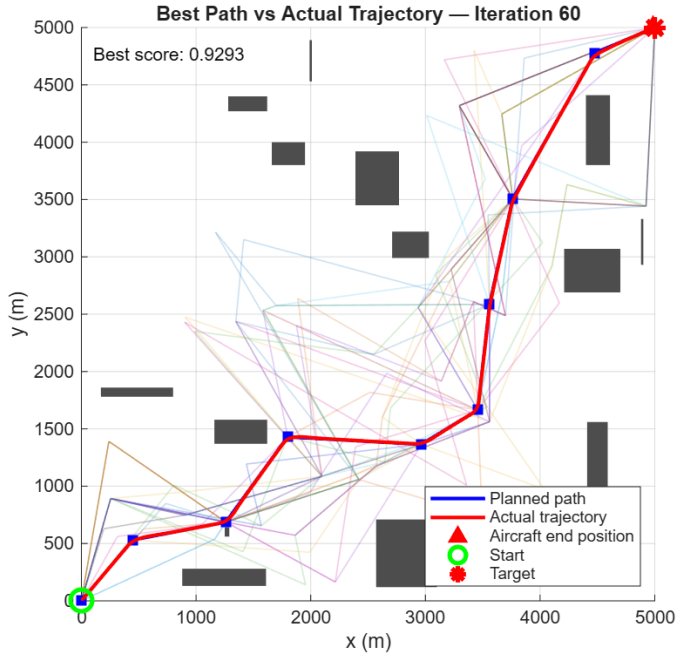

Fig. 10. Population and best path after 60 iterations.

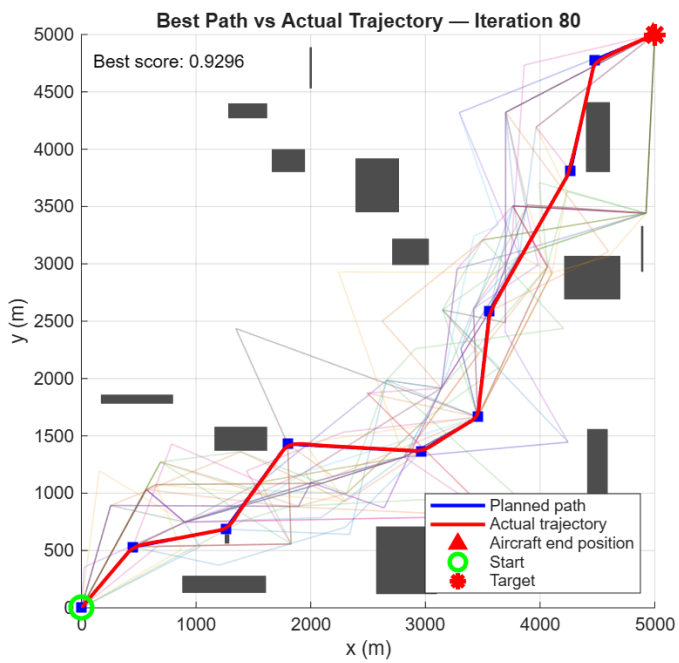

Fig. 11. Population and best path after 80 iterations.

An important implementation consideration was population size. Early testing with a population of 20 individuals resulted in noticeably slower convergence and a higher likelihood of the algorithm becoming trapped in suboptimal solutions. Increasing the population size to 40 provided sufficient diversity to reliably explore the environment and converge within approximately 40 iterations. This improvement comes at the cost of increased computational expense, as each additional individual requires a full nonlinear simulation for fitness evaluation. As a result, the selected population size represents a tradeoff between solution quality and computational time.

B. Simulation 2: Time-Optimized Cost Function (No Throttle Penalty)

To analyze the impact of the throttle penalty on the resulting flight path, a second simulation was conducted. In this iteration, the throttle scoring function was completely removed, and its corresponding weight was redistributed to the time score. This creates an aggressive cost function that strictly prioritizes reaching the target as quickly as possible, regardless of the energy expenditure required.

Fig. 12 provides a direct visual comparison of the best paths generated by both cost functions after 50 iterations. Inspection of the wind field overlaid on the figure reveals that the dominant wind direction across the domain is oriented toward the lower-right, meaning that flight toward the upper-right target is generally conducted into a crosswind. The two paths respond to this condition in noticeably different ways.

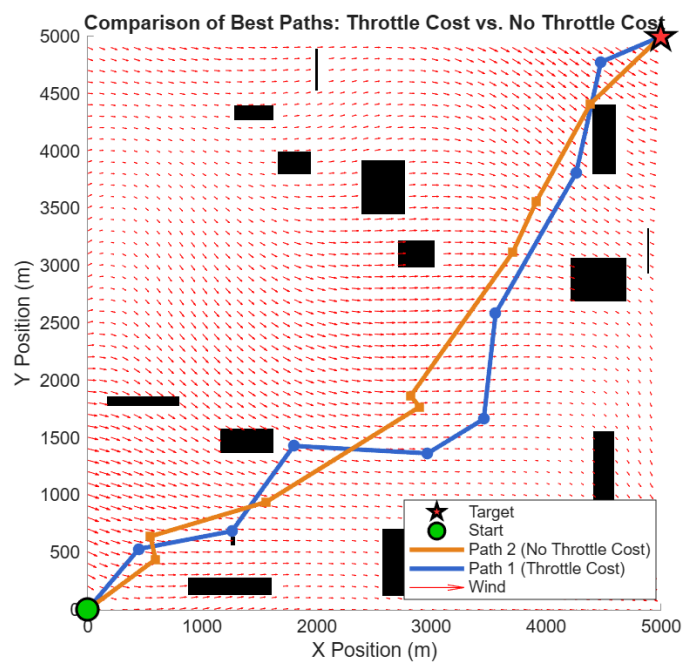

Fig. 12. Comparison of the best paths after 50 iterations using Cost Function 1 (Throttle Penalty) and Cost Function 2 (Time-Optimized).

Path 1, produced by the throttle-penalized cost function, takes a more deliberate route through the domain. Rather than tracking the diagonal directly, it deviates laterally at several

points, particularly in regions where the wind magnitude is highest. These deviations indicate that the algorithm is identifying more favorable wind corridors, either by avoiding strong headwinds or exploiting localized tailwinds. This results in a longer geometric path, but one that requires less sustained throttle input.

Path 2, produced by the time-optimized cost function, takes a more direct route toward the target. Without any penalty on throttle usage, the algorithm is free to fly aggressively into headwind regions if doing so reduces total flight time. The resulting path is geometrically shorter and follows the diagonal more closely, but does so at the cost of increased control effort.

Table II quantifies these behavioral differences. Because the two cost functions assign different weights to their respective components, the total scores are not directly comparable across formulations. Instead, the individual component scores, before weighting, provide the meaningful basis for comparison. Path achieves both a higher time score and a higher throttle score than Path 2. This confirms that the cost of throttle is working to find a more efficient route. The more surprising result is the higher time score in Path 1, meaning this more efficient route also helps the aircraft save time overall as well. Both paths achieve a perfect obstacle avoidance score, demonstrating that the EA consistently converges to collision-free solutions. This simulation was repeated on different environments produced by the same speed, similar results were shown over each variation. Both cost functions were able to correctly avoid obstacles. The first cost function consistently produced a lower overall throttle use. While the time score was very similar between both cost functions in every simulation. This trend was observed consistently across multiple environments, reinforcing that the inclusion of a throttle penalty produces a physically meaningful and repeatable change in planner behavior without noticeably affecting the overall time of flight.

TABLE II
PERFORMANCE COMPARISON: THROTTLE PENALTY VS. TIME-OPTIMIZED COST FUNCTIONS.

Metric	**CF1 (Throttle)**	**CF2 (No Throttle)**
Time Score	0.9966	0.9965
Throttle Score	0.8198	0.8075
Obstacle Score	1.0000	1.0000

IV. CONCLUSION

This paper presented the design, implementation, and evaluation of an Evolutionary Algorithm for path planning of a fixed-wing sUAS operating in a steady ambient wind field with static obstacles. The central goal was to determine whether explicitly incorporating throttle usage into the EA fitness function produces meaningfully more energy-efficient flight paths compared to a baseline planner that optimizes for time and obstacle avoidance alone.

The results demonstrate that the proposed wind-aware cost function successfully guides the population toward collision-free trajectories, achieving a perfect obstacle avoidance score of 1.0 by convergence. Compared to the time-optimized baseline, the throttle-penalized cost function produced a path with a meaningfully higher throttle score, confirming that the throttle fitness term influences the algorithm to select routes that reduce engine demand by avoiding sustained wind exposure. Time of flight remained competitive across both formulations, indicating that the throttle penalty does not significantly compromise route efficiency.

The current implementation has several limitations. A consistent behavioral pattern observed across both cost functions is a tendency to route waypoints in close proximity to obstacle boundaries. While these paths technically satisfy the obstacle avoidance constraint as evaluated against the simulated trajectory, the narrow clearances produced would present an unacceptable collision risk in a real operational environment, where wind gusts, tracking error, and trajectory following imprecision would likely push the aircraft into the structure. Future work should introduce a spatial buffer penalty into the fitness function that continuously discourages waypoints from approaching within a minimum safe standoff distance from any obstacle boundary, producing paths that are not only technically collision-free but operationally safe.

The algorithm as presented operates in a two-dimensional horizontal plane, which precludes the evaluation of altitude-dependent wind that could be significant for a sUAS. Extending the framework to a full three-dimensional simulation environment would open the possibility of explicitly modeling and exploiting vertical wind structures such as updrafts and thermals. For a platform like the Ttwistor, which is designed to operate in severe weather environments, the ability to plan routes that extract energy from updrafts could meaningfully extend mission endurance, and represents a natural next step for a wind-aware cost function that already penalizes throttle usage.

Finally, while the computational cost of full nonlinear dynamics simulation at every generation makes the current implementation best suited to pre-mission ground-based planning over large domains, a reduction in environment size and population could make the algorithm viable for onboard real-time obstacle avoidance in more constrained operational scenarios. A smaller planning horizon combined with a simplified dynamics model would reduce the per-generation evaluation cost substantially, potentially enabling replanning within the time constraints of a low-altitude urban flight. Validating this capability, along with hardware-in-the-loop or flight test evaluation on the Ttwistor platform, would be necessary steps toward operational deployment of the algorithm.

REFERENCES

[1] J. da Silva Arantes, M. da Silva Arantes, C. F. M. Toledo, and B. C. Williams, "A Multi-Population Genetic Algorithm for UAV Path Re-Planning under Critical Situation," in *Proc. 27th IEEE Int. Conf. Tools Artif. Intell. (ICTAI)*, Vietri sul Mare, Italy, Nov. 2015, pp. 486–493. doi: 10.1109/ICTAI.2015.75.

[2] Y. Jiang, X.-X. Xu, M.-Y. Zheng, and Z.-H. Zhan, "Evolutionary Computation for Unmanned Aerial Vehicle Path Planning: A Survey," *Artif. Intell. Rev.*, vol. 57, no. 10, p. 267, Oct. 2024. doi: 10.1007/s10462-024-10913-0.

[3] D. Rathbun, S. Kragelund, A. Pongpunwattana, and B. Capozzi, "An Evolution Based Path Planning Algorithm for Autonomous Motion of a UAV through Uncertain Environments," in *Proc. 21st Digital Avionics Syst. Conf.*, vol. 2, Oct. 2002, pp. 8D2–8D2. doi: 10.1109/DASC.2002.1052946.

[4] D. Rathbun and B. Capozzi, "Evolutionary Approaches to Path Planning Through Uncertain Environments," presented at the 1st UAV Conf., May 2002. doi: 10.2514/6.2002-3455.

[5] J. Xiao, Z. Michalewicz, L. Zhang, and K. Trojanowski, "Adaptive Evolutionary Planner/Navigator for Mobile Robots," *IEEE Trans. Evol. Comput.*, vol. 1, no. 1, pp. 18–28, Apr. 1997. doi: 10.1109/4235.585889.

[6] T. Sayyah *et al.*, "Wind and the City: Utilizing UAV-Based In-Situ Measurements for Estimating Urban Wind Fields," *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 11094–11101.

[7] Y. Zhang *et al.*, "Research on impact of range resolutions on wind detection," *Optics Express*, vol. 33, no. 15, 2025.

[8] U.S. Department of Defense, "Unmanned Systems Integrated Roadmap FY2013-2038," *DoD Report*, Dec. 2013, p. 5.

[9] T. Oke, "Street design and urban canopy layer climate," *Energy and Buildings*, vol. 11, no. 1-3, pp. 103–113, 1988.

[10] G. de Boer *et al.*, "Evaluation and Intercomparison of Small Uncrewed Aircraft Systems Used for Atmospheric Research," *J. Atmos. Oceanic Technol.*, vol. 41, no. 2, Feb. 2024.

[11] J. Ware and N. Roy, "An analysis of wind field estimation and exploitation for quadrotor flight in urban environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Stockholm, Sweden, May 2016, pp. 2487–2494. doi: 10.1109/ICRA.2016.7487401.

APPENDIX

The full MATLAB implementation of the evolutionary path planning algorithm, including the environment generator and fitness evaluation scripts, is available at this repository.